Snoluthando Siziba

Software developer CV

Software Development Student | Junior Developer Profile

DOCUMENT PURPOSE

A consolidated working record of software-development education, technical skills, practical projects, tools, and related computing knowledge discussed to date. It is designed as a foundation for an e-portfolio, technical CV, graduate applications, and interviews. Review project status and proficiency labels before sharing externally.

EDUCATION	CORE AREAS	CAREER DIRECTION
CTU Training Solutions	**Web development • Python**	**Junior software development**
OCC Software Developer	**SQL • Application development**	**Data / cloud / AI pathways**

1. Professional Overview

Software development student at CTU Training Solutions, building practical experience across responsive web applications, database-backed systems, business platforms, and software problem-solving. Work to date combines hands-on implementation with coursework in software-development processes, databases, cloud computing, cybersecurity, artificial intelligence, Internet of Things, and modern digital systems.

The strongest self-identified technical strength is debugging and troubleshooting. Practical work has involved diagnosing frontend/backend integration issues, deployment configuration, authentication, database connectivity, and local development environment problems. The user also makes extensive use of AI-assisted workflows and prompt engineering as productivity tools, while continuing to build independent implementation depth.

POSITIONING STATEMENT

An emerging software developer with practical exposure to end-to-end web application delivery, relational databases, API integrations, version control, and business-oriented software. Particularly interested in building lean, useful systems that solve operational problems rather than adding unnecessary complexity.

2. Education & Learning

- Institution: CTU Training Solutions.
- Programme / study context: OCC Software Developer.
- Coursework and learning areas discussed: programming, web development, databases, software testing and debugging, mobile applications, cloud computing, AI, IoT, big data, cybersecurity, and Microsoft collaboration tools.
- Software Development Life Cycle (SDLC): requirements collection, feasibility analysis, planning, implementation, testing, deployment, and maintenance.
- Development methodologies: Waterfall and Agile; has evaluated when an iterative Agile approach is appropriate, including stakeholder feedback and incremental feature delivery.
- Requirements analysis: functional versus non-functional requirements, stakeholder needs, and translating business problems into system features.
- Design and planning: wireframes, application structure, user experience, and practical feature scoping.
- Professional skills covered: teamwork, communication, problem-solving, and technical career awareness.

3. Technical Skills Inventory

The table separates practical use from learning or exploration. It is intentionally conservative: familiarity does not automatically imply professional-level proficiency.

Technology / area	Experience level	Evidence / application
HTML5	Intermediate	Page structure, semantic content, web layouts.
CSS3	Intermediate	Responsive styling, layout, visual hierarchy, presentation.
JavaScript	Intermediate	Browser-side behaviour, interaction, form logic, frontend functionality.
Python	Intermediate	Application logic, Flask development, scripting and technical experimentation.
SQL	High intermediate	Relational data, queries, CRUD operations, database-backed applications.
C#	Basic	Repository and application work in a .NET environment; continue building depth.
Dart / Flutter	Low intermediate	Sentinel mobile-app concept and Flutter-related work.
TypeScript	Low intermediate	Considered as a primary language for modern web application development.
React / Node.js / PHP	Minimal	

SQL Server	Intermediate	Database connectivity, tables, queries, SQL Server Management Studio, ODBC.
Supabase	Well experienced	Database-backed app development, authentication/profile data, Edge Functions.
Git / GitHub	Intermediate	Repositories, commits, remotes, branches, pull/push workflows, troubleshooting.
Vercel / Netlify	Well experienced	Web application hosting and deployment workflows.
Power BI	Intermediate	Business intelligence and reporting coursework / tool use.
Azure	Basic	Cloud learning; AZ-204 identified as a planned certification pathway.
Flask	Basic	Python web application routes, sessions, templates, redirects and flash messages.
APIs / integrations	Low intermediate	Application integrations, email workflows, payment integration planning and implementation exposure.
MT5 / Python automation	Low intermediate	Trading-bot logic and strategy automation experimentation.

4. Tools, Platforms & Development Environment

- Visual Studio Code as a primary code editor / IDE.
- PowerShell and Windows command-line workflows for project setup, package installation, Git operations, and environment troubleshooting.
- Git and GitHub for source control, repository management, collaboration workflows, and remote hosting.
- Supabase for application data and authentication-related functionality; Supabase Edge Functions considered/used for server-side integrations.
- SQL Server and SQL Server Management Studio (SSMS), including ODBC connectivity.
- Vercel and Netlify for web deployment and hosting.
- Azure learning and Microsoft cloud ecosystem exposure.
- Python virtual environments and package management; exposure to OpenCV / MediaPipe dependency setup in a computer-vision project.
- EmailJS and payment-provider integration concepts, including PayFast and Peach Payments evaluation.

5. Selected Projects & Practical Experience

Flow — Small-Business SaaS Ecosystem

Personal product ecosystem / active development | A family of lightweight business applications intended to share credentials and data while addressing separate operational workflows.

Technology / concepts: Next.js, TypeScript direction, Tailwind CSS, Supabase, SQL, GitHub, Vercel / Netlify concepts.

- InvoiceFlow: invoicing for small businesses and freelancers, focused on customer/product records, invoice creation, totals, PDF download, email workflows, invoice status, and dashboard statistics. It is positioned as invoicing software rather than a full accounting package.
- InvoiceFlow data model discussed includes a Supabase profiles table with UUID id, nullable full_name, created_at defaulting to now, and role defaulting to user.
- Invoice lifecycle includes draft, sent, viewed, paid, overdue, and cancelled; overdue can be derived from due date and stored status rather than treated as an independent manual state.
- Release work has included making dashboard status cards functional and allowing users to open invoices and update status from the dashboard or invoice page.
- OrderFlow: planned next product in the ecosystem; intended to capture orders, persist records, email clients, and support stock / logistics visibility.
- SpendFlow: planned purchase and spend-tracking product; branch card purchases and scanned slips feed head-office records and branch-level business intelligence.

- Design preference: lightweight, maintainable applications with shared foundations; avoid unnecessary abstraction and excessive component/file fragmentation.

Sentinel — Public Safety / Patrol Awareness App

Concept / project exploration | Flutter mobile-app concept intended to improve public and patrol awareness through a shared map experience for public users, police, and private security.

Technology / concepts: Flutter / Dart, maps and geolocation concepts, role-based application workflows.

- Default map centre was specified around Johannesburg (-26.0274, 28.0088).
- Concept includes public-facing awareness and more restricted police / private-security functionality.
- The concept demonstrates interest in location-aware mobile systems and civic technology.

RomaGym — Gym Membership Management System

Coursework / practical Flask application | A Python web application for managing gym users and members, with authentication and member-management workflows.

Technology / concepts: Python, Flask, SQL Server, pyodbc, ODBC Driver 17 for SQL Server, HTML templates.

- Implemented routes and patterns including sessions, login, dashboard, logout, redirects, flash messages, and templates.
- Database work involved Users and Members tables, member registration/listing, active-member counts, and seeded sample member records.
- Demonstrates relational database connectivity and CRUD-style application logic.

Manners Africa — Clothing Brand Website

Client project / redesign | A refined e-commerce-oriented brand website for a contemporary clothing label, with emphasis on a restrained visual identity and a maintainable implementation.

Technology / concepts: HTML, CSS, JavaScript, responsive UI, payment integration evaluation, PayFast, Supabase Edge Functions.

- Client approved the overall layout and requested no major layout changes; the implementation direction prioritises a lighter, simpler codebase.
- Brand direction: quiet luxury / minimal contemporary essentials; palette references include void, bone, ash, and stamp red.
- Typography direction discussed: Anton, Space Grotesk, and JetBrains Mono.
- Payment-provider evaluation included PayFast versus Peach Payments; PayFast sandbox and South African shipping configuration were discussed.

Envision Works — Business Website

Client / portfolio project | Business website work for an organisation operating around Mokopane, Limpopo, and Gauteng.

Technology / concepts: HTML, CSS, JavaScript, EmailJS.

- Work focused on website structure, responsive presentation, visual design, and business-facing communication.
- Email functionality was identified as an outstanding integration item during a prospective client demonstration / sales discussion.

WeBuyApple — Website / Business Platform

Earlier project / redesign experience | Website redesign and business-platform work focused on structure, responsiveness, user experience, and visual presentation.

Technology / concepts: Web technologies, database integration and API-driven functionality (specific implementation details vary by project stage).

- Payment-system integration was part of the broader project experience discussed, including PayFast and Supabase Edge Functions.

MineOps — .NET Application Repository

Repository / development environment work | A software project maintained in a GitHub repository under the BlackrockSoftware organisation.

Technology / concepts: .NET / C#, Git, .NET SDK and local development tooling.

- Worked through SDK version and environment alignment, including .NET 8 SDK versions and planned installation of additional SDK versions.
- Repository: BlackrockSoftware/MineOps.

SMC — Python / MetaTrader 5 Trading Bot

Personal automation project | Algorithmic trading-bot concept using market structure and multi-timeframe analysis.

Technology / concepts: Python, MetaTrader 5, XM broker environment, technical-analysis concepts.

- Markets discussed include US100, US30, XAUUSD, with GER40 as a desired addition.
- Risk framework discussed: 1% risk per trade, stop after two weekly losses, and a 10% monthly target.
- Strategy outline: 4-hour context, point of interest (POI), then 15-minute order-block (OB) execution concept.
- These are strategy design parameters, not evidence of verified trading performance.

6. Software Engineering & Computing Knowledge

Software Development Life Cycle

- Requirements: identify users, stakeholders, business objectives, constraints, and expected system behaviour.
- Feasibility: assess technical feasibility, cost, operational fit, schedule, and risks before committing to a solution.
- Design: plan architecture, data structures, user flows, interfaces, and integration points.
- Implementation: write and integrate code in manageable increments.
- Testing: verify expected behaviour, validate edge cases, troubleshoot defects, and perform regression checks.
- Deployment and maintenance: release the system, monitor issues, support users, and improve the product over time.

Agile and Waterfall

- Waterfall is a sequential approach in which phases are typically completed in order; it can suit projects with stable, well-defined requirements.
- Agile uses iterative delivery, prioritised work, frequent feedback, and adaptation as requirements evolve.
- Agile was considered suitable in coursework for a SpazaConnect scenario, where stock-tracking and product-management features could be delivered incrementally and refined through stakeholder feedback.

Databases & Data

- Relational database fundamentals: tables, records, columns, primary keys, foreign keys, relationships, and structured queries.
- CRUD operations: create, read, update, and delete records through application interfaces and database queries.
- Application data modelling: designing tables around users, profiles, customers, products, invoices, members, and operational records.
- Database connectivity: SQL Server through pyodbc / ODBC and Supabase-backed application data.
- Business intelligence concepts: reporting, operational metrics, dashboards, branch-level spend analysis, and Power BI exposure.

Web & Application Architecture

- Frontend: HTML structures content, CSS controls presentation and responsive layouts, and JavaScript adds client-side behaviour.

- Backend: application routes and server-side logic handle requests, business rules, authentication flows, and integrations.
- APIs: allow systems to exchange data and invoke functionality; integration reliability depends on correct request handling, credentials, validation, and error handling.
- Authentication and sessions: establish user identity and maintain authenticated application state; authorisation determines which actions a user may perform.
- Responsive design and UX: adapt layouts to different screen sizes and make important tasks discoverable and usable.
- Deployment: source code, environment variables, build configuration, hosting, domains, and backend services must be configured consistently.

Cloud, AI, IoT & Cybersecurity

- Cloud computing: on-demand access to shared computing resources and services; coursework and learning include cloud concepts and Microsoft Azure.
- AI and machine learning: coursework-level understanding of how data-driven models can support prediction, classification, recommendations, and automation; Python is a relevant implementation language.
- Internet of Things: connected devices and sensors can collect and transmit data for monitoring, automation, and decision support.
- Big data: large or complex datasets create requirements around storage, processing, analysis, and visualisation.
- Cybersecurity: core awareness of protecting systems, identities, data, and communications; practical application should include secure authentication, least privilege, input validation, secret management, and safe handling of user data.
- These are documented learning and conceptual areas; the document does not imply specialist-level production experience in AI, cloud architecture, IoT, or cybersecurity.

Testing, Debugging & Technical Problem-Solving

- Debugging is the strongest self-identified technical capability.
- Practical troubleshooting has covered frontend/backend integration, deployment configuration, authentication, database connectivity, package dependencies, SDK versions, Git remotes/branches, and local environment setup.
- Useful testing habits include reproducing the issue, narrowing the cause, checking logs and error messages, validating assumptions, testing the fix, and checking for regressions.
- A maintainable solution should favour clear naming, readable functions, minimal duplication, appropriate separation of concerns, and complexity proportionate to the problem.

7. Business & Product Thinking

- Translating operational problems into small, practical software products, as seen in the Flow ecosystem and SpendFlow concept.
- Understanding user roles and workflows, including business owners, customers, branch staff, and administrators.
- Prioritising minimum viable functionality and incremental releases rather than overbuilding.
- Considering payment providers, email communication, hosting costs, maintenance, and ongoing support as part of delivering a usable business website or application.
- Interest in small-business software, operational dashboards, civic technology, and systems that reduce manual administration.

8. Certifications & Development Roadmap

- Pearson IT Specialist: Python
- Pearson IT Specialist: HTML and CSS
- Microsoft Azure coursework completed / undertaken; AZ-204 (Developing Solutions for Microsoft Azure) identified as a planned certification.
- Continue strengthening TypeScript and modern full-stack application development.

- Build deeper evidence in API design, automated testing, secure authentication, deployment pipelines, and production monitoring.
- Expand cloud implementation experience through Azure projects and a completed AZ-204 learning path.
- For AI / data pathways, build demonstrable Python data analysis and machine-learning projects rather than relying only on theoretical familiarity.

9. Career Interests & Target Roles

- Junior Software Developer / Graduate Software Developer.
- Web Application Developer / Full-Stack Developer (junior pathway).
- SQL Developer / Database-oriented development.
- Data Science Developer and AI / Machine Learning engineering pathways.
- Cloud Graduate Programme / Azure-focused development.
- Longer-term interest in advanced AI / ML research and technically demanding engineering environments, including defence-adjacent technical work.

10. Working Style & Technical Strengths

- Strongest self-reported area: debugging and systematic troubleshooting.
- Uses AI tools and prompt engineering extensively to accelerate research, problem-solving, and development; generated output is treated as something to inspect, test, and refine.
- Prefers lightweight, understandable project structures and practical implementation over unnecessary architectural complexity.
- Interested in visual quality and user experience as well as functionality.
- Business-minded approach: evaluates whether a feature solves a real user or operational need.

11. Contact details

- Linkedin- www.linkedin.com/in/snoluthando-siziba-9a760b350
- Github-. https://github.com/Snoluthando
- E-portfolio-
- Email- sf.siziba@gmail.com
- Phone number – 083 952 0098/ 083 747 1121

Appendix A — Technical Vocabulary Reference

Term	Meaning
API	Application Programming Interface; a defined way for software systems to communicate.
CRUD	Create, Read, Update, Delete; the basic operations used to manage stored records.
DBMS	Database Management System; software used to store, organise, query, and manage data.
Git	Distributed version-control system used to track source-code changes.
GitHub	A platform for hosting Git repositories and collaborating on software.
IDE	Integrated Development Environment; software that supports coding, debugging, and project workflows.
MVP	Minimum Viable Product; the smallest useful version of a product that can be tested with users.
ODBC	Open Database Connectivity; a standard interface for applications to connect to database systems.

ORM	Object-Relational Mapper; a tool that maps application objects to relational database structures.
REST	A common architectural style for web APIs using resources and standard HTTP methods.
RLS	Row-Level Security; database rules that restrict which rows a user can access.
SDLC	Software Development Life Cycle; the process of planning, building, testing, releasing, and maintaining software.
UI / UX	User Interface / User Experience; the interface presented to users and the quality of their interaction.
CI/CD	Continuous Integration / Continuous Delivery or Deployment; automated workflows for building, testing, and releasing software.
Environment variables	Configuration values supplied to an application at runtime, often used for service URLs and secrets.